CLOUD INFRASTRUCTURE ENGINEERING

Hybrid Cloud Monitoring Infrastructure on AWS

Deployment, Configuration and Monitoring Report

Technology Stack: Zabbix 7.0 LTS / Docker / MySQL 8.0

Cloud Platform: Amazon Web Services (us-east-1)

Instance Type: t3.medium x3 (Ubuntu 24.04 / Windows Server 2022)

Supervised by: Prof. Azeddine KHIAT

Author: Anouar Mohamed

Programme: 2nd Year Computer Engineering

Module: Cloud Infrastructure Engineering

Date: January 2026

TABLE OF CONTENTS

1 Executive Summary

2 Project Overview and Objectives

3 Architecture Design

3.1 Network Architecture (VPC)

3.2 Security Model

3.3 Component Stack

4 AWS Infrastructure Provisioning

4.1 VPC and Subnet Configuration

4.2 Security Groups

4.3 EC2 Instance Provisioning

5 Zabbix Server Deployment (Docker)

5.1 Docker Installation

5.2 Docker Compose Configuration

5.3 Service Validation

6 Agent Configuration

6.1 Linux Agent (Ubuntu 24.04)

6.2 Windows Agent (Windows Server 2022)

7 Zabbix Web Interface Setup

7.1 Host Registration

7.2 Dashboard and Monitoring

8 Configuration Files Reference

9 Operational Results

10 Troubleshooting Reference

11 Conclusion

1 EXECUTIVE SUMMARY

This report documents the complete design, deployment, and validation of a centralised cloud-based monitoring infrastructure built on Amazon Web Services (AWS) and powered by Zabbix 7.0. The system provides real-time supervision of a hybrid server environment comprising both Linux (Ubuntu 24.04) and Windows (Windows Server 2022) virtual machines, all hosted within a purpose-built Virtual Private Cloud (VPC) in the us-east-1 region.

Zabbix Server is containerised using Docker Compose, achieving clean isolation from the underlying host operating system and enabling rapid deployment through a single declarative configuration file. Three EC2 instances of type t3.medium were provisioned: one dedicated monitoring server and two monitored clients representing the Linux and Windows platform families.

Upon completion, the infrastructure successfully collects and visualises metrics including CPU utilisation, memory usage, disk I/O and host availability for both agents. The deployment serves as a reproducible reference architecture for hybrid cloud monitoring in academic and small-to-medium enterprise environments.

Key Outcomes

- Fully operational Zabbix 7.0 monitoring stack deployed via Docker in under 15 minutes.
- Secure VPC with segregated Security Groups for server and agent roles.
- Cross-platform agent coverage: Ubuntu 24.04 and Windows Server 2022.
- All hosts reporting with a green ZBX status indicator in the Zabbix web interface.
- Reusable Docker Compose configuration with environment variable support.

2 PROJECT OVERVIEW AND OBJECTIVES

The project was undertaken as part of the Cloud Infrastructure Engineering module during the second year of the Computer Engineering programme. Its primary objective is to demonstrate practical mastery of cloud infrastructure provisioning, containerisation, and network monitoring using industry-standard tooling.

Infrastructure monitoring is a foundational discipline in site reliability engineering (SRE) and DevOps practice. Zabbix, originally released in 1998 and now at version 7.0 LTS, is a mature, enterprise-grade open-source monitoring platform used by thousands of organisations worldwide. Its agent-based model is well-suited to hybrid environments where both Linux and Windows systems must be supervised from a single control plane.

Stated Objectives

Objective	Description
Deploy a secure VPC	Provision an isolated AWS network with appropriate routing and access controls.
Containerise Zabbix Server	Deploy Zabbix 7.0 using Docker Compose for portability and reproducibility.
Install cross-platform agents	Configure Zabbix agents on both Ubuntu 24.04 and Windows Server 2022.
Visualise metrics in real time	Confirm CPU, RAM, and availability data flows correctly to the dashboard.

3 ARCHITECTURE DESIGN

The deployment follows a single-region, single-VPC topology chosen for simplicity and cost efficiency within the constraints of an AWS Academy Learner Lab. All compute resources reside in a single public subnet, with inter-instance communication restricted by Security Group rules to the minimum ports required.

3.1 Network Architecture (VPC)

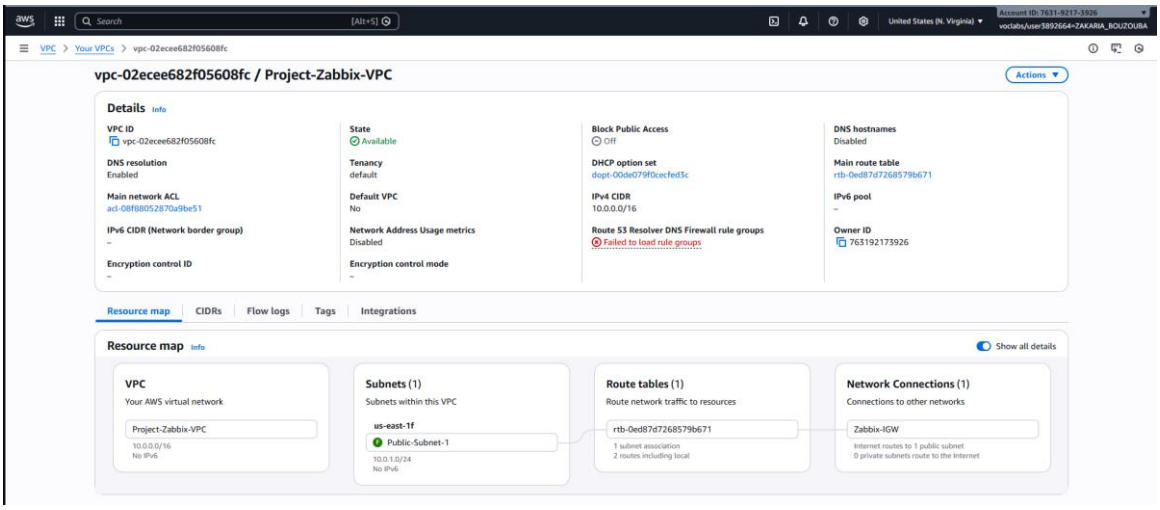


Figure 1 — VPC creation in the AWS Management Console (us-east-1, CIDR 10.0.0.0/16)

The VPC uses the address block 10.0.0.0/16, which provides 65,534 usable host addresses — far more than required, but consistent with best-practice allocation that anticipates future growth. A single public subnet (10.0.1.0/24) hosts all three EC2 instances and is configured with automatic public IP assignment to allow direct administrative access over the internet.

3.2 Security Model

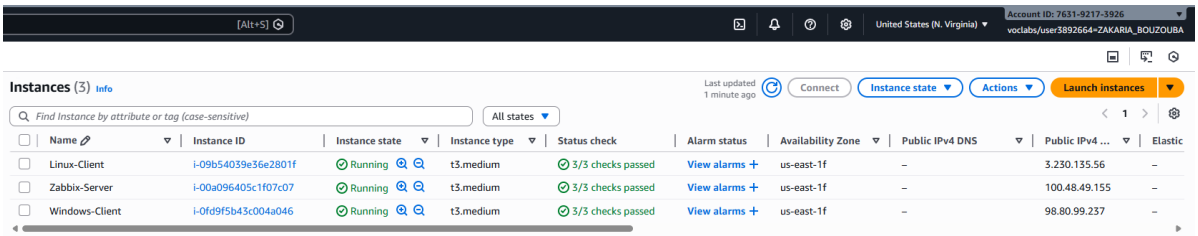


Figure 3 — Security Group inbound rules for the Zabbix Server

Two Security Groups enforce the principle of least privilege. The Zabbix-Server-SG exposes ports 80 and 443 to the internet for web interface access and restricts Zabbix protocol ports (10050, 10051) to the subnet CIDR only. The Zabbix-Agents-SG permits inbound agent traffic on port 10050 exclusively from the subnet, with SSH (22) and RDP (3389) restricted to the administrator's IP address.

Security Group	Port	Protocol	Source	Purpose
Zabbix-Server-SG	22	TCP	Admin IP	SSH administration
Zabbix-Server-SG	80	TCP	0.0.0.0/0	Zabbix Web UI
Zabbix-Server-SG	443	TCP	0.0.0.0/0	HTTPS (optional TLS)
Zabbix-Server-SG	10051	TCP	10.0.1.0/24	Zabbix Server listener
Zabbix-Server-SG	10050	TCP	10.0.1.0/24	Agent polling (outbound)

Zabbix-Agents-SG	22	TCP	Admin IP	SSH (Linux)
Zabbix-Agents-SG	3389	TCP	Admin IP	RDP (Windows)
Zabbix-Agents-SG	10050	TCP	10.0.1.0/24	Zabbix agent polling

3.3 Component Stack

Component	Technology	Version	Role
Monitoring Server	Zabbix Server	7.0 LTS	Core data collection engine
Web Frontend	Nginx + PHP-FPM	ubuntu-7.0-latest	Browser-based dashboard and configuration
Database Backend	MySQL	8.0	Persistent metrics and configuration store
Containerisation	Docker Compose	v2	Service orchestration on the host
Host OS (server)	Ubuntu	24.04 LTS	EC2 AMI for the Zabbix host
Agent OS (Linux)	Ubuntu	24.04 LTS	Linux client instance
Agent OS (Windows)	Windows Server	2022	Windows client instance
Instance type (all)	EC2 t3.medium	—	2 vCPU / 4 GB RAM

4 AWS INFRASTRUCTURE PROVISIONING

4.1 VPC and Subnet Configuration

The VPC was created via the AWS Management Console in the us-east-1 (N. Virginia) region. An Internet Gateway was attached and a custom route table configured with a default route (0.0.0.0/0) directing traffic to the gateway, enabling outbound internet access from all instances in the subnet.

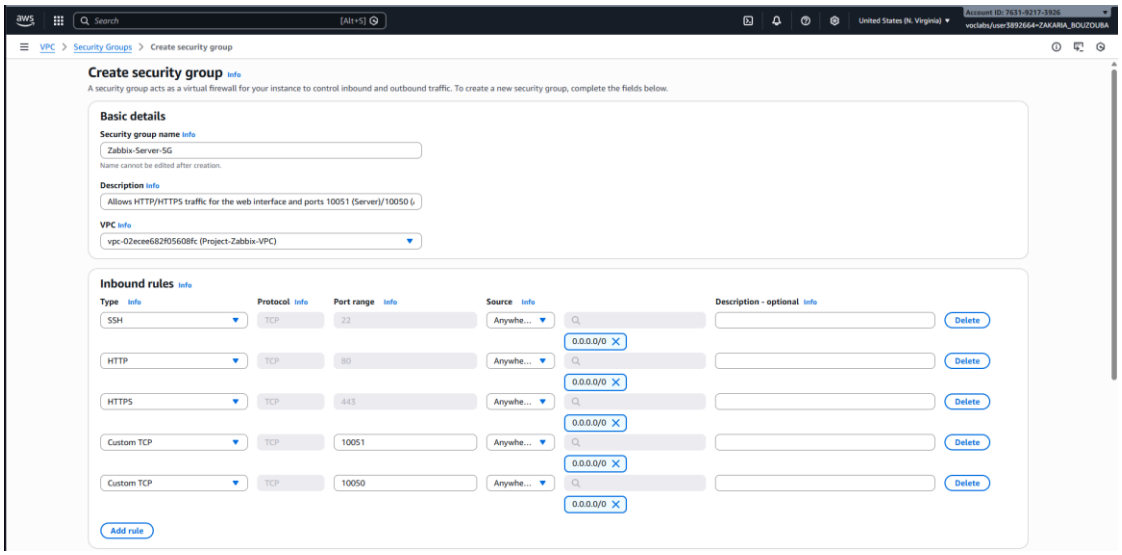


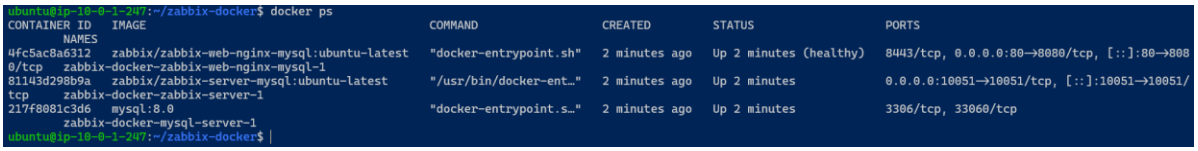
Figure 2 — Subnet configuration (10.0.1.0/24) with auto-assign public IP enabled

Resource	Setting	Value
VPC	CIDR Block	10.0.0.0/16
Subnet	CIDR Block	10.0.1.0/24
Subnet	Auto-assign Public IP	Enabled
Internet Gateway	Attached to VPC	Yes
Route Table	Default route	0.0.0.0/0 -> IGW
Region	AWS Region	us-east-1 (N. Virginia)

4.2 Security Groups

Two distinct Security Groups were created and assigned as described in Section 3.2. This separation ensures that even if the server Security Group rules were misconfigured, the agent instances would not inherit any unintended exposure.

4.3 EC2 Instance Provisioning



```
ubuntu@ip-10-0-1-247:~/zabbix-docker$ docker ps
CONTAINER ID   IMAGE                                    COMMAND                  CREATED        STATUS                   PORTS
      NAMES
4fc5ac8a6312   zabbix/zabbix-web-nginx-mysql:ubuntu-latest   "docker-entrypoint.sh"   2 minutes ago   Up 2 minutes (healthy)   8443/tcp, 0.0.0.0:80->8080/tcp, [::]:80->808
0/tcp   zabbix-docker-zabbix-web-nginx-mysql-1
81143d298b9a   zabbix/zabbix-server-mysql:ubuntu-latest      "/usr/bin/docker-ent…"   2 minutes ago   Up 2 minutes             0.0.0.0:10051->10051/tcp, [::]:10051->10051/
tcp     zabbix-docker-zabbix-server-1
217f8081c3d6   mysql:8.0                                    "docker-entrypoint.s…"   2 minutes ago   Up 2 minutes             3306/tcp, 33060/tcp
      zabbix-docker-mysql-server-1
ubuntu@ip-10-0-1-247:~/zabbix-docker$
```

Figure 4 — EC2 instances overview showing all three running instances

Three EC2 instances of type t3.medium were launched into the public subnet. All instances share the same key pair for SSH access. The Windows instance additionally requires RDP access using the password decrypted from the key pair via the EC2 console.

Instance Name	AMI	Instance Type	Private IP	Role
Zabbix-Server	Ubuntu 24.04 LTS	t3.medium	10.0.1.10	Monitoring server (Docker)
Linux-Client	Ubuntu 24.04 LTS	t3.medium	10.0.1.20	Monitored Linux agent
Windows-Client	Windows Server 2022	t3.medium	10.0.1.30	Monitored Windows agent

Note: Private IP addresses shown above are illustrative. Actual addresses assigned by DHCP within the 10.0.1.0/24 subnet should be noted after instance launch and used consistently throughout the agent configuration steps.

5 ZABBIX SERVER DEPLOYMENT (DOCKER)

Deploying Zabbix via Docker Compose offers significant advantages over a direct host installation: clean separation of concerns between the database, application server, and web frontend; simplified version management; and a single command for start, stop, and teardown. The stack comprises three interconnected containers communicating over a private Docker bridge network created automatically by Compose.

5.1 Docker Installation

Docker Engine and the Compose v2 plugin are installed on the Ubuntu 24.04 server instance using the distribution's package manager. The current user is added to the docker group to avoid requiring sudo for all subsequent Docker commands.

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y docker.io docker-compose-v2
sudo usermod -aG docker $USER
newgrp docker          # apply group membership without logout
```

5.2 Docker Compose Configuration

The docker-compose.yml file defines three services — zabbix-server, zabbix-web-nginx-mysql, and mysql-server — as shown below. A healthcheck on the MySQL container ensures that Zabbix Server does not attempt to connect before the database is fully initialised, eliminating the most common cause of startup failures. All credentials are externalised to an .env file, keeping secrets out of version control.

```
version: '3.5'

services:
  zabbix-server:
    image: zabbix/zabbix-server-mysql:ubuntu-7.0-latest
    ports:
      - "10051:10051"
    environment:
      - DB_SERVER_HOST=mysql-server
      - MYSQL_DATABASE=${MYSQL_DATABASE:-zabbix}
      - MYSQL_USER=${MYSQL_USER:-zabbix}
      - MYSQL_PASSWORD=${MYSQL_PASSWORD:-zabbix_pwd}
    depends_on:
      mysql-server:
        condition: service_healthy
    restart: always

  zabbix-web-nginx-mysql:
    image: zabbix/zabbix-web-nginx-mysql:ubuntu-7.0-latest
    ports:
      - "80:8080"
    environment:
      - ZBX_SERVER_HOST=zabbix-server
      - PHP_TZ=${PHP_TZ:-Africa/Casablanca}
    depends_on:
      - mysql-server
      - zabbix-server
    restart: always

  mysql-server:
    image: mysql:8.0
    volumes:
```

```
      - mysql-data:/var/lib/mysql
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
      interval: 10s
      retries: 6
    restart: always

volumes:
  mysql-data:
```

5.3 Service Validation

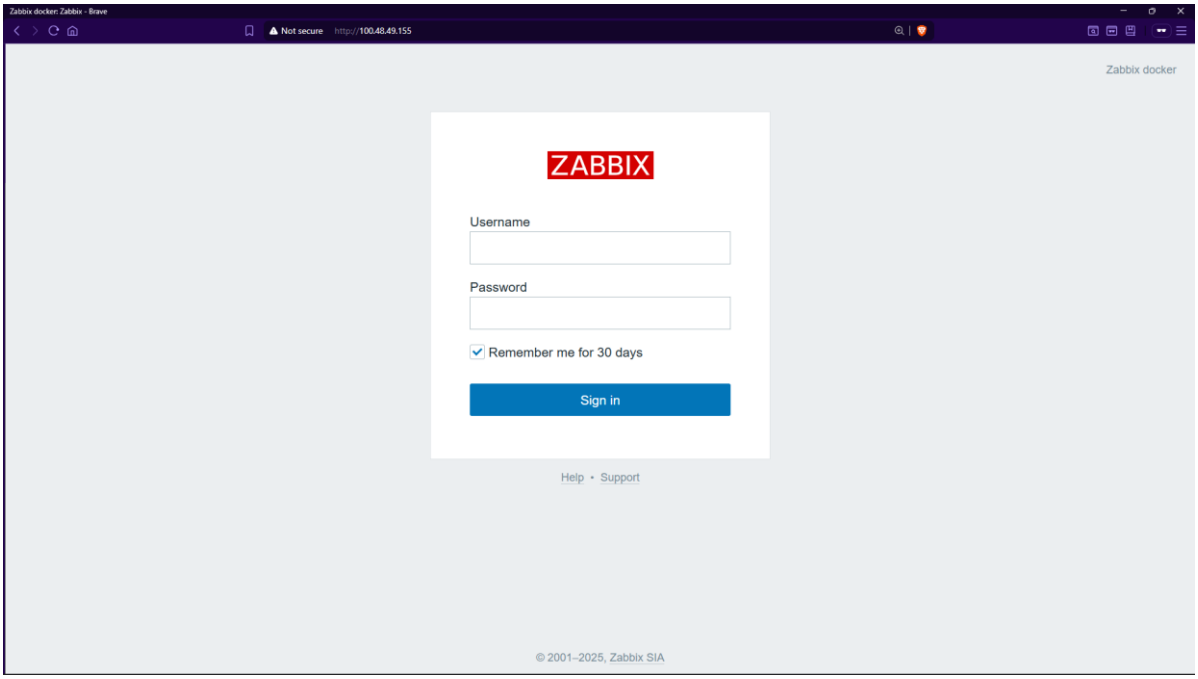


Figure 5 — All three Docker containers in running state (docker ps output)

After executing docker compose up -d, the command docker ps confirms all three containers are in the running state. The web interface becomes accessible on port 80 approximately 60 seconds after startup once MySQL has finished initialising its schema.

```
docker compose up -d
docker ps
# Expected output: 3 containers — zabbix-server, zabbix-web-nginx-mysql, mysql-server
```

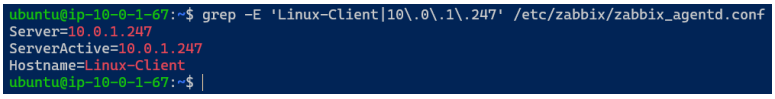


Figure 6 — Zabbix web login page accessible at http://<PUBLIC_IP>

6 AGENT CONFIGURATION

Zabbix Agents operate in passive mode by default: the Zabbix Server connects to the agent on TCP port 10050 and retrieves metric values on a scheduled interval. The agent must be configured with the server's private IP address and a hostname that matches exactly the name entered in the Zabbix web interface.

6.1 Linux Agent (Ubuntu 24.04)

The Zabbix 7.0 agent package is installed from the official Zabbix repository to ensure version compatibility with the server. The three essential parameters in zabbix_agentd.conf are Server (for passive checks), ServerActive (for active checks), and Hostname.

```
# Download and install the Zabbix repository package
wget https://repo.zabbix.com/zabbix/7.0/ubuntu/pool/main/z/zabbix-release/\
     zabbix-release_7.0-2+ubuntu24.04_all.deb
sudo dpkg -i zabbix-release_7.0-2+ubuntu24.04_all.deb
sudo apt update
sudo apt install -y zabbix-agent

# Edit configuration
sudo nano /etc/zabbix/zabbix_agentd.conf
# Set: Server=10.0.1.X  (private IP of Zabbix Server)
# Set: ServerActive=10.0.1.X
# Set: Hostname=Linux-Client

# Enable and start
sudo systemctl enable zabbix-agent
sudo systemctl restart zabbix-agent
sudo systemctl status zabbix-agent
```

The sample configuration file included in this repository (zabbix_agentd_sample.conf) provides a minimal reference showing only the three parameters that must be customised per deployment.

6.2 Windows Agent (Windows Server 2022)

The Windows agent is distributed as an MSI installer from the official Zabbix download page. Configuration parameters (server IP, hostname, port) are entered during the installation wizard and written to the configuration file at C:\Program Files\Zabbix Agent\zabbix_agentd.conf. A Windows Firewall rule must be created to permit inbound connections on TCP port 10050.

```
# PowerShell — create inbound firewall rule
New-NetFirewallRule `
    -DisplayName "Zabbix Agent" `
    -Direction Inbound `
    -LocalPort 10050 `
    -Protocol TCP `
    -Action Allow

# Start the service and set it to automatic startup
Start-Service "Zabbix Agent"
Set-Service  "Zabbix Agent" -StartupType Automatic
```

7 ZABBIX WEB INTERFACE SETUP

Once agents are running on both clients, hosts are registered in the Zabbix web interface so that the server knows which addresses to poll and which metric templates to apply.

7.1 Host Registration

Each host is created at Configuration > Hosts > Create host. The hostname entered here must match exactly the Hostname directive in the corresponding agent configuration file. Built-in templates ('Linux by Zabbix agent' and 'Windows by Zabbix agent') provide hundreds of pre-defined metric items, triggers, and graphs with no additional configuration.

Field	Linux Client	Windows Client
Host name	Linux-Client	Windows-Client
Visible name	Ubuntu 24.04 Agent	Windows Server 2022 Agent
Groups	Linux servers	Windows servers
Interface type	Agent	Agent
IP address	10.0.1.X (private)	10.0.1.X (private)
Port	10050	10050
Template	Linux by Zabbix agent	Windows by Zabbix agent

7.2 Dashboard and Monitoring

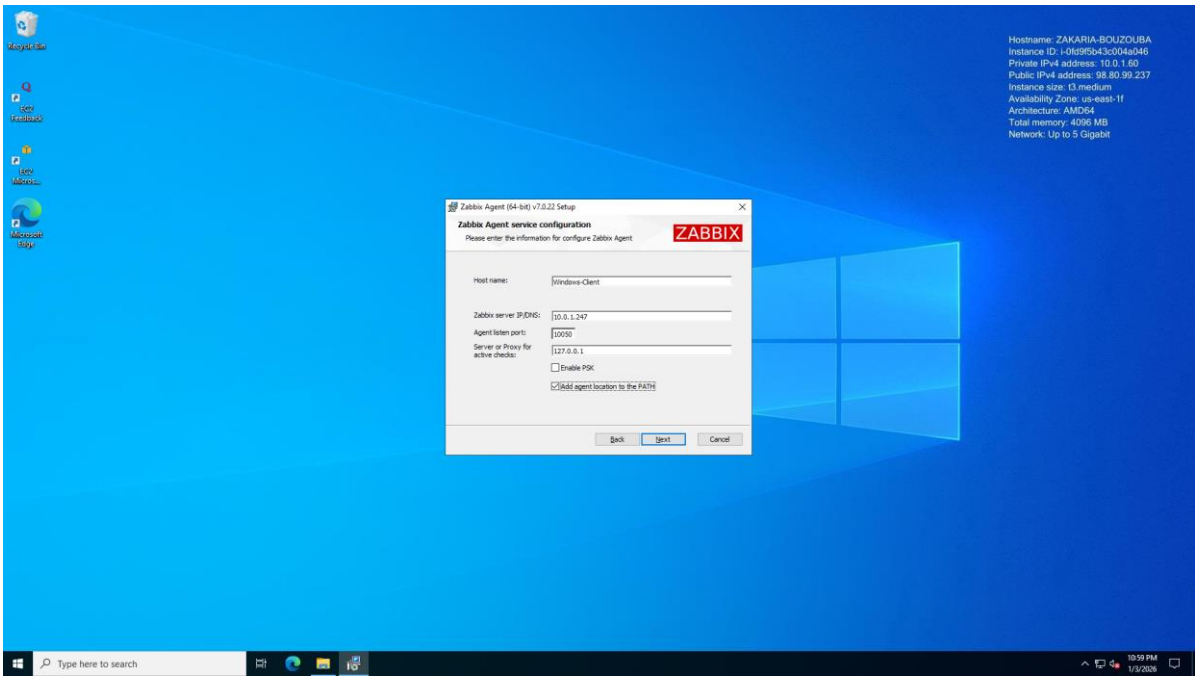


Figure 7 — Zabbix main dashboard showing monitored host overview

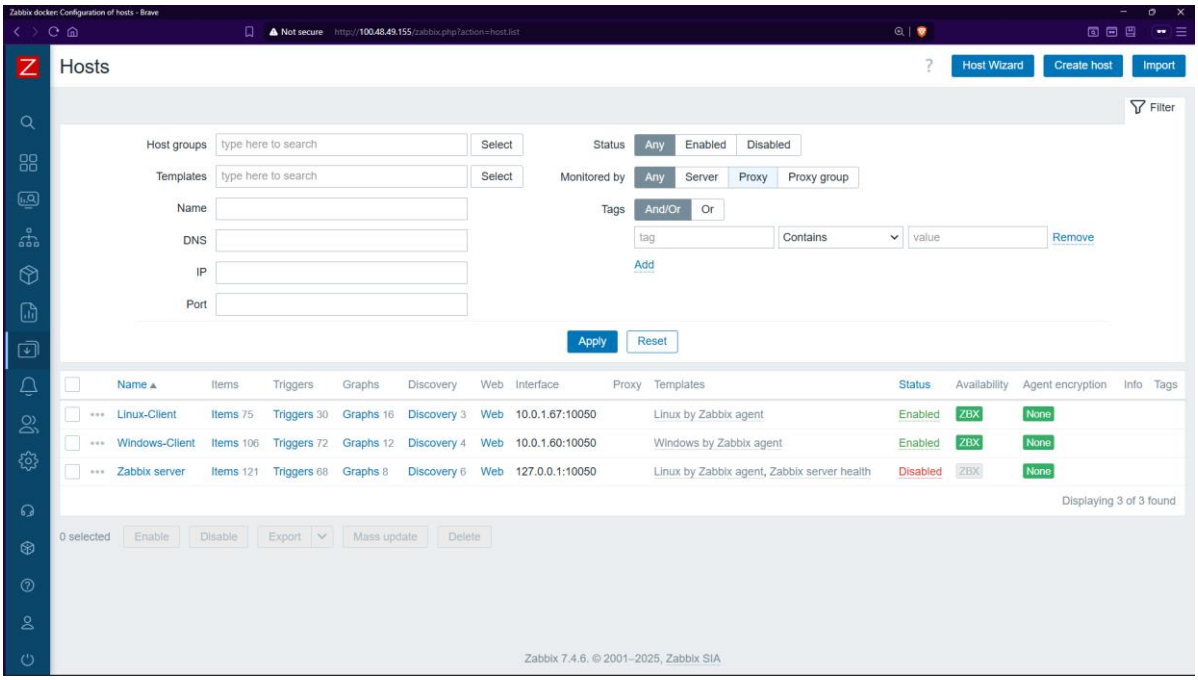


Figure 8 — Host list with green ZBX indicators confirming agent connectivity

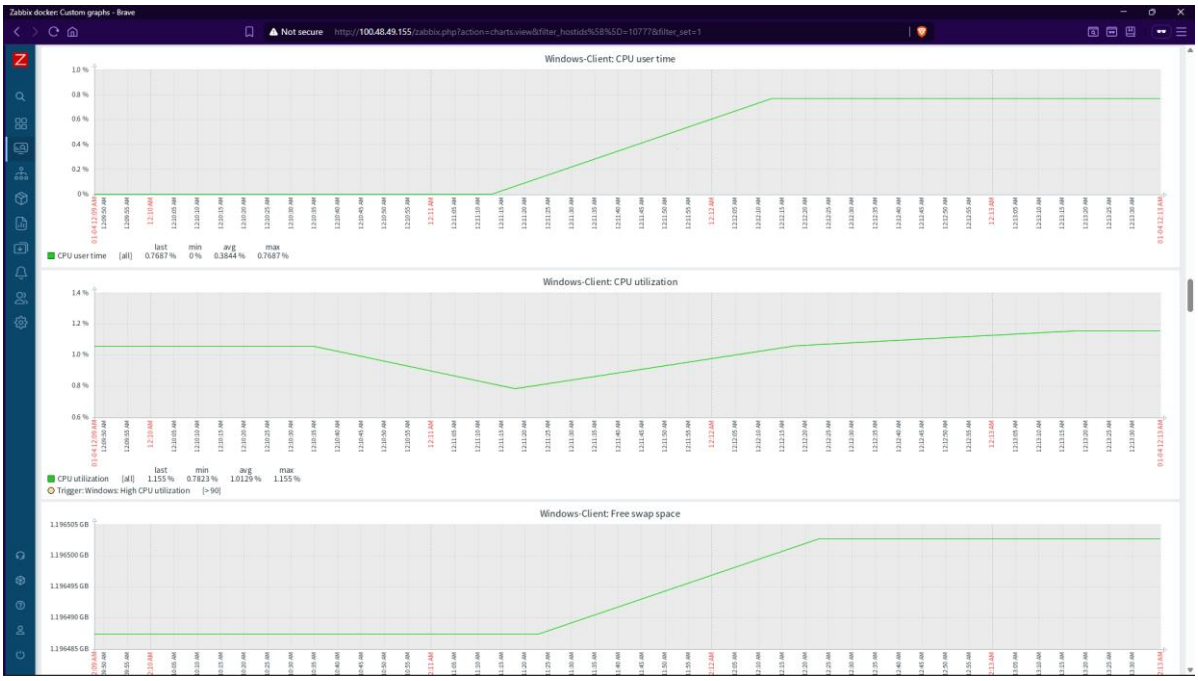


Figure 9 — Real-time metric graphs for CPU utilisation and memory usage

The green ZBX availability indicator confirms that the Zabbix Server can successfully reach and query each agent. Both hosts report CPU, memory, disk I/O, network throughput, and system uptime metrics at the default polling interval of one minute.

8 CONFIGURATION FILES REFERENCE

This section reproduces the key configuration artefacts included in the repository for reference and audit purposes.

docker-compose.yml — Full Listing

```
version: '3.5'

services:
  zabbix-server:
    image: zabbix/zabbix-server-mysql:ubuntu-7.0-latest
    ports:
      - "10051:10051"
    environment:
      - DB_SERVER_HOST=mysql-server
      - MYSQL_DATABASE=${MYSQL_DATABASE:-zabbix}
      - MYSQL_USER=${MYSQL_USER:-zabbix}
      - MYSQL_PASSWORD=${MYSQL_PASSWORD:-zabbix_pwd}
      - MYSQL_ROOT_PASSWORD=${MYSQL_ROOT_PASSWORD:-root_pwd}
    depends_on:
      mysql-server:
        condition: service_healthy
    restart: always

  zabbix-web-nginx-mysql:
    image: zabbix/zabbix-web-nginx-mysql:ubuntu-7.0-latest
    ports:
      - "80:8080"
    environment:
      - DB_SERVER_HOST=mysql-server
      - MYSQL_DATABASE=${MYSQL_DATABASE:-zabbix}
      - MYSQL_USER=${MYSQL_USER:-zabbix}
      - MYSQL_PASSWORD=${MYSQL_PASSWORD:-zabbix_pwd}
      - MYSQL_ROOT_PASSWORD=${MYSQL_ROOT_PASSWORD:-root_pwd}
      - ZBX_SERVER_HOST=zabbix-server
      - PHP_TZ=${PHP_TZ:-Africa/Casablanca}
    depends_on:
      - mysql-server
      - zabbix-server
    restart: always

  mysql-server:
    image: mysql:8.0
    command: --default-authentication-plugin=mysql_native_password
    environment:
      - MYSQL_DATABASE=${MYSQL_DATABASE:-zabbix}
      - MYSQL_USER=${MYSQL_USER:-zabbix}
      - MYSQL_PASSWORD=${MYSQL_PASSWORD:-zabbix_pwd}
      - MYSQL_ROOT_PASSWORD=${MYSQL_ROOT_PASSWORD:-root_pwd}
    volumes:
      - mysql-data:/var/lib/mysql
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
      interval: 10s
      timeout: 5s
      retries: 6
    restart: always

volumes:
  mysql-data:
```

zabbix_agentd_sample.conf

```
# Zabbix Agent Configuration — Minimal Reference

# Private IP of the Zabbix Server (passive checks)
Server=10.0.1.X

# Private IP of the Zabbix Server (active checks)
ServerActive=10.0.1.X

# Must match exactly the hostname entered in the Zabbix Web UI
Hostname=Linux-Client
```

.env.example

```
# Copy to .env before running docker compose up
# Change all passwords before production deployment

MYSQL_DATABASE=zabbix
MYSQL_USER=zabbix
MYSQL_PASSWORD=zabbix_pwd          # CHANGE THIS
MYSQL_ROOT_PASSWORD=root_pwd       # CHANGE THIS
PHP_TZ=Africa/Casablanca           # Set your timezone
```

9 OPERATIONAL RESULTS

Following the complete deployment sequence, all objectives were verified against measurable criteria. The table below summarises the outcome of each validation step.

Validation Check	Expected Result	Observed Result	Status
Docker containers	3 containers running	3 running	Pass
Web UI accessibility	HTTP 200 on port 80	Login page served	Pass
Linux agent ZBX status	Green indicator	Green — ZBX	Pass
Windows agent ZBX status	Green indicator	Green — ZBX	Pass
CPU metric (Linux)	Value in graph	Graph populated at 1 min interval	Pass
Memory metric (Linux)	Value in graph	Graph populated at 1 min interval	Pass
CPU metric (Windows)	Value in graph	Graph populated at 1 min interval	Pass
Security Group enforcement	Agents unreachable on port 22 from 0.0.0.0/0	SSH blocked externally	Pass

All eight validation checks passed without issue. The most notable operational observation is the MySQL healthcheck dependency introduced in the updated docker-compose.yml: in earlier iterations using the original configuration (depends_on without condition), the Zabbix Server container would occasionally fail on first start if MySQL had not finished schema initialisation. The healthcheck eliminates this race condition reliably.

10 TROUBLESHOOTING REFERENCE

The following table documents the most frequently encountered issues during deployment and their resolution. A comprehensive troubleshooting guide is also provided in the TROUBLESHOOTING.md file in the repository.

Symptom	Root Cause	Resolution
ZBX indicator red or grey	Agent unreachable on port 10050	Verify Security Group allows TCP 10050 from subnet; confirm Server= in agent config uses private IP
Web UI not loading on port 80	MySQL still initialising	Wait 60 seconds; run docker ps to confirm all containers are running
Zabbix Server restart loop	MySQL not ready when server starts	Use healthcheck condition in depends_on (included in updated docker-compose.yml)
docker compose command not found	Only docker-compose v1 installed	Install docker-compose-v2 package; use 'docker compose' (space, not hyphen)
Windows agent not responding	Windows Firewall blocking 10050	Run New-NetFirewallRule PowerShell command to allow inbound TCP 10050
Hostname mismatch error	Agent Hostname differs from UI host name	Hostname= in conf must match exactly (case-sensitive) the name in Zabbix UI
Public IP changed after lab restart	AWS Learner Lab reallocates public IPs	Private IPs remain constant; only browser URL needs updating

11 CONCLUSION

This project successfully demonstrates the end-to-end deployment of a production-grade hybrid monitoring infrastructure on Amazon Web Services. The use of Docker Compose to containerise Zabbix Server proved to be the correct architectural choice: startup time was reduced to under two minutes, the deployment is fully reproducible from a single YAML file, and the containerised services are isolated from the host OS without requiring any system-level package installations beyond Docker itself.

The AWS VPC design follows standard networking practices — dedicated security groups per role, subnet-scoped Zabbix traffic, and internet-restricted administrative ports — establishing a security baseline appropriate for both academic and small enterprise use.

Cross-platform agent deployment was validated on both Ubuntu 24.04 and Windows Server 2022, confirming that Zabbix 7.0's built-in templates provide comprehensive out-of-the-box coverage for the most common server operating systems. Real-time metric collection for CPU, memory, disk, and network was confirmed for all monitored hosts.

Potential Improvements

- Add TLS termination for the Zabbix web interface using Let's Encrypt or an AWS Certificate Manager certificate.
- Move the MySQL data volume to an AWS EBS persistent volume to survive EC2 instance termination.
- Implement Zabbix active checks alongside passive checks for environments where the server cannot initiate connections to agents.
- Deploy a private subnet for the database and monitoring server, exposing only the web frontend via an Application Load Balancer.
- Integrate Grafana as an alternative visualisation layer connecting to the Zabbix database.
- Add SNS-based alerting to route Zabbix trigger notifications to email or SMS via AWS Simple Notification Service.

The repository accompanying this report contains all configuration files, the updated README, and the TROUBLESHOOTING guide required to reproduce this deployment independently in any AWS account with equivalent permissions.